CS224N Lecture 6: LSTM RNNs and Neural Machine Translation

Stanford University

Spring 2024

Abstract

This lecture continues the study of Recurrent Neural Networks (RNNs) by introducing Long Short-Term Memory (LSTM) networks, which address the vanishing gradient problem in standard RNNs. We explore the architecture and mechanisms of LSTMs, examine various RNN applications, and introduce Neural Machine Translation as a sequence-to-sequence learning task.

Contents

1 Review: Language Models and RNNs

1.1 Language Models

Definition 1.1 (Language Model)**.** A **language model** is a system that predicts the next word in a sequence given the preceding context. Formally, it computes:

$$P(w_t \mid w_1, w_2, \ldots, w_{t-1})$$

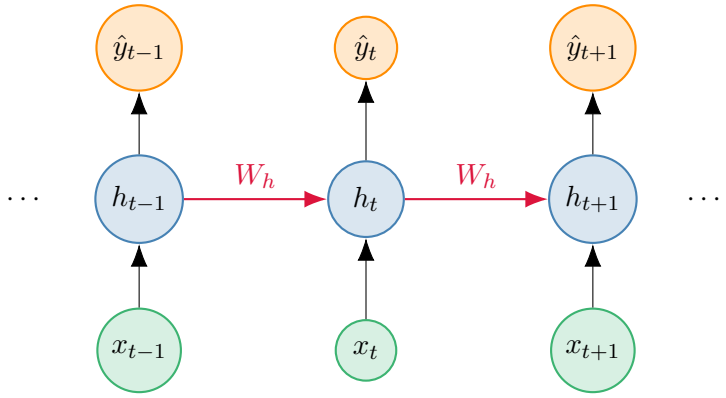
Language models are fundamental to many NLP tasks including:

- Text generation
- Speech recognition
- Machine translation
- Text likelihood estimation

1.2 Recurrent Neural Networks (RNNs)

RNNs are neural architectures designed to process sequential input of arbitrary length by:

- Applying the same weights at each time step
- Maintaining a hidden state that carries information through the sequence
- Optionally producing output at each time step



The RNN update equations are:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b) \tag{1}$$
$$\hat{y}_t = \text{softmax}(W_y h_t + b_y) \tag{2}$$

where σ is typically tanh or ReLU activation function.

2 Evaluating Language Models

2.1 Perplexity

Definition 2.1 (Perplexity)**.** **Perplexity** is the standard metric for evaluating language models, defined as:

$$\text{Perplexity} = \sqrt[N]{\prod_{t=1}^{N} \frac{1}{P(w_t \mid w_1, \ldots, w_{t-1})}}$$

where N is the number of words in the test sequence.

Relationship to Cross-Entropy

Perplexity is the exponential of the cross-entropy:

$$\text{Perplexity} = \exp\left(-\frac{1}{N}\sum_{t=1}^{N}\log P(w_t \mid w_1, \ldots, w_{t-1})\right)$$

Lower perplexity is better.

Remark 2.1. Perplexity can be interpreted as the effective number of equally-likely choices at each position. For example, perplexity of 64 means the model is as uncertain as if choosing uniformly from 64 words.

2.2 Historical Progress in Language Modeling

Model Type	**Perplexity**
Early n-gram models	∼150
Kneser-Ney smoothed n-grams	67
RNN + MaxEnt hybrid	51
LSTM models	30–43
Modern transformers	<10

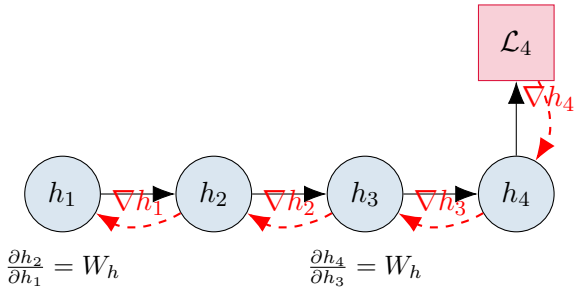
Table 1: Evolution of language model perplexity on standard benchmarks

3 The Vanishing and Exploding Gradient Problem

3.1 Problem Formulation

When training RNNs via backpropagation through time (BPTT), we compute gradients by repeatedly applying the chain rule:

$$\frac{\partial \mathcal{L}_t}{\partial h_k} = \frac{\partial \mathcal{L}_t}{\partial h_t} \cdot \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$$



3.2 Mathematical Analysis

Under the simplification that σ is the identity function:

$$\frac{\partial h_t}{\partial h_{t-1}} = W_h$$

Therefore:

$$\frac{\partial \mathcal{L}_t}{\partial h_k} \propto W_h^{t-k}$$

Using eigenvalue decomposition $W_h = Q\Lambda Q^{-1}$:

$$W_h^{t-k} = Q\Lambda^{t-k}Q^{-1}$$

Critical Issue

- If all eigenvalues $|\lambda_i| < 1$: **Vanishing gradients**
  $\Rightarrow$ Gradients decay exponentially with distance
- If any eigenvalue $|\lambda_i| > 1$: **Exploding gradients**
  $\Rightarrow$ Gradients grow exponentially with distance

3.3 Consequences

3.3.1 Vanishing Gradients

- Gradient signal from distant time steps becomes negligibly small
- Model cannot learn long-range dependencies
- Effective context window ≈ 7 tokens for standard RNNs
- Updates dominated by local context

Example 3.1 (Long-Distance Dependency)**.** Consider the sentence:

> "When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her ___."

A human easily predicts "**tickets**" (23 words back), but standard RNNs fail because gradient signal doesn't propagate that far.

3.3.2 Exploding Gradients

- Very large gradients lead to enormous parameter updates
- Can cause NaN/Inf values
- Training becomes unstable
- Model parameters diverge

3.4 Solution: Gradient Clipping

For exploding gradients, the standard solution is **gradient clipping**:

Algorithm 1 Gradient Clipping

```
Compute gradient g
Compute gradient norm ||g||
if ||g|| > threshold then
    g ← threshold · g/||g||
end if
Apply gradient update with clipped g
```

Typical threshold values: 5–20.

Remark 3.1. Gradient clipping is a simple heuristic but highly effective in practice and widely used in training deep networks.

4 Long Short-Term Memory (LSTM) Networks

4.1 Motivation

The core problem with vanilla RNNs is that the hidden state is completely rewritten at each time step:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

This makes it difficult to:

- Preserve information over long sequences
- Control what information to remember or forget
- Prevent vanishing/exploding gradients

Key Insight: Design an architecture with explicit memory and gating mechanisms.

4.2 LSTM Architecture

Definition 4.1 (LSTM)**.** A **Long Short-Term Memory (LSTM)** network (Hochreiter & Schmidhuber, 1997; Gers et al., 2000) is a type of RNN with:

- A **cell state** c_t that acts as long-term memory
- A **hidden state** h_t for output and computation
- Three **gates** that control information flow

4.2.1 LSTM Gates

All gates are computed similarly using sigmoid activation:

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f) \quad \text{(Forget gate)} \tag{3}$$
$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i) \quad \text{(Input gate)} \tag{4}$$
$$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o) \quad \text{(Output gate)} \tag{5}$$

where σ is the logistic function: $\sigma(z) = \frac{1}{1+e^{-z}}$, producing values in $[0, 1]$.

- **Forget gate** f_t: Controls how much of previous cell state to retain
- **Input gate** i_t: Controls how much new information to add
- **Output gate** o_t: Controls how much cell state to expose

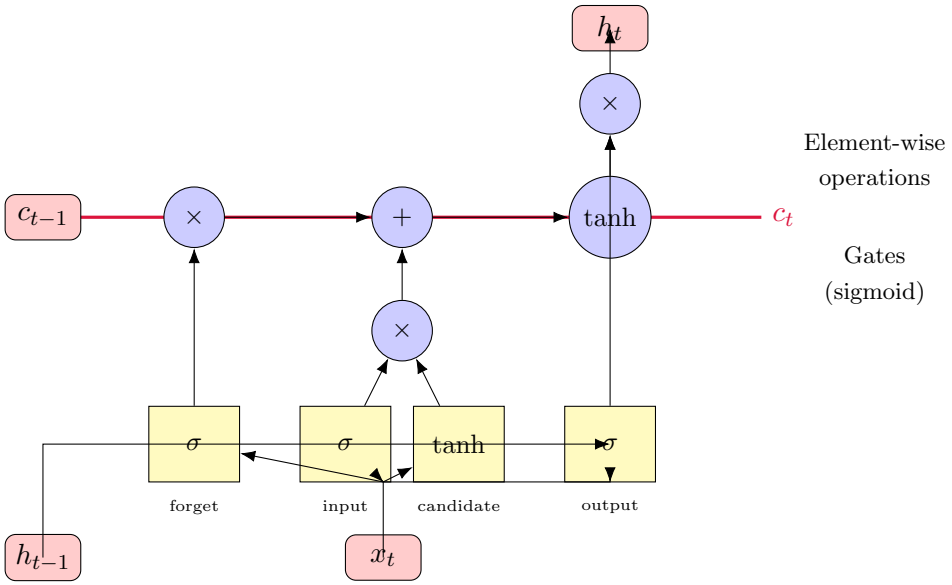
4.2.2 LSTM Update Equations

LSTM Update Rules

$$\tilde{c}_t = \tanh(W_c h_{t-1} + U_c x_t + b_c) \quad \text{(Candidate cell state)} \tag{6}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{(New cell state)} \tag{7}$$
$$h_t = o_t \odot \tanh(c_t) \quad \text{(New hidden state)} \tag{8}$$

where $\odot$ denotes element-wise (Hadamard) product.

4.3 LSTM Diagram



4.4 Why LSTMs Work

4.4.1 Gradient Flow

The key innovation is the **additive** update for the cell state:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Theorem 4.1 (LSTM Gradient Flow)**.** The gradient flowing backwards through the cell state is:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

which is controlled by the forget gate rather than a repeated matrix multiplication.

Advantages of LSTM Architecture

- **Additive updates** prevent vanishing gradients
- Setting $f_t = 1, i_t = 0$ allows perfect information preservation
- No repeated matrix multiplications $\Rightarrow$ no eigenvalue issues
- Can learn to store information for arbitrary time periods

4.4.2 Information Control

- **Forget gate**: Model learns which information to discard
  - May forget when encountering new topics, contexts
  - Example: Forget subject information after seeing end-of-sentence
- **Input gate**: Model learns which new information is relevant
  - May pay attention to named entities, key verbs
  - Ignore filler words, common adjectives
- **Output gate**: Model learns what information to output
  - Separate long-term storage from generation
  - Example: Store "King of Prussia" but don't output until relevant

4.5 Historical Context

- Original LSTM (Hochreiter & Schmidhuber, 1997): No forget gate
- Improved LSTM (Gers et al., 2000): Added forget gate
- Little attention until mid-2000s (Alex Graves' work)
- Breakthrough adoption 2014–2016 via Google
- Dominant architecture for sequence modeling until transformers

4.6 LSTM vs Standard RNN Comparison

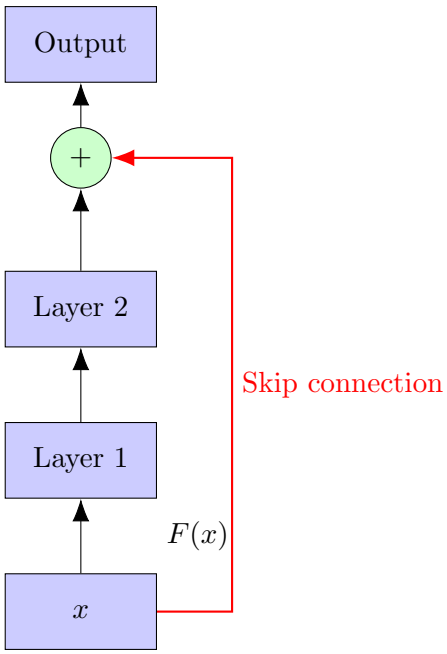
Property	**Standard RNN**	**LSTM**
Hidden state update	Multiplicative	Additive
Gradient flow	Exponential decay/growth	Gated control
Effective context	$\sim$7 tokens	$>$100 tokens
Parameters	$O(n^2)$	$O(4n^2)$
Vanishing gradient	Severe	Mitigated

5 Advanced RNN Architectures

5.1 Solutions to Vanishing Gradients in Deep Networks

The vanishing gradient problem is not unique to RNNs—it affects any deep network. Several architectural solutions exist:

5.1.1 Residual Networks (ResNets)



Residual connection: $y = F(x) + x$

- Identity mapping allows gradient to flow directly backwards
- Layer learns residual $F(x)$ rather than full transformation
- Revolutionized computer vision (He et al., 2015)

5.1.2 Dense Networks (DenseNets)

Connect each layer to all subsequent layers:

$$x_\ell = H_\ell([x_0, x_1, \ldots, x_{\ell-1}])$$

where $[\cdot]$ denotes concatenation.

5.1.3 Highway Networks

Gated residual connections (analogous to LSTM gates):

$$y = T(x) \odot F(x) + (1 - T(x)) \odot x$$

where $T(x)$ is a learned transform gate.

5.2 Bidirectional RNNs

Motivation

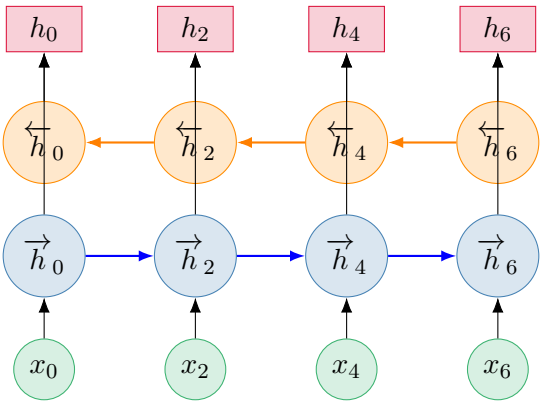
Standard RNNs only have access to *past* context. For many tasks (e.g., tagging, classification), we want to use *both past and future* context.

Definition 5.1 (Bidirectional RNN)**.** A **Bidirectional RNN** processes the sequence in both directions and concatenates the hidden states:

$$\overrightarrow{h}_t = \text{RNN}_{\text{forward}}(x_t, \overrightarrow{h}_{t-1}) \tag{9}$$

$$\overleftarrow{h}_t = \text{RNN}_{\text{backward}}(x_t, \overleftarrow{h}_{t+1}) \tag{10}$$

$$h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t] \tag{11}$$

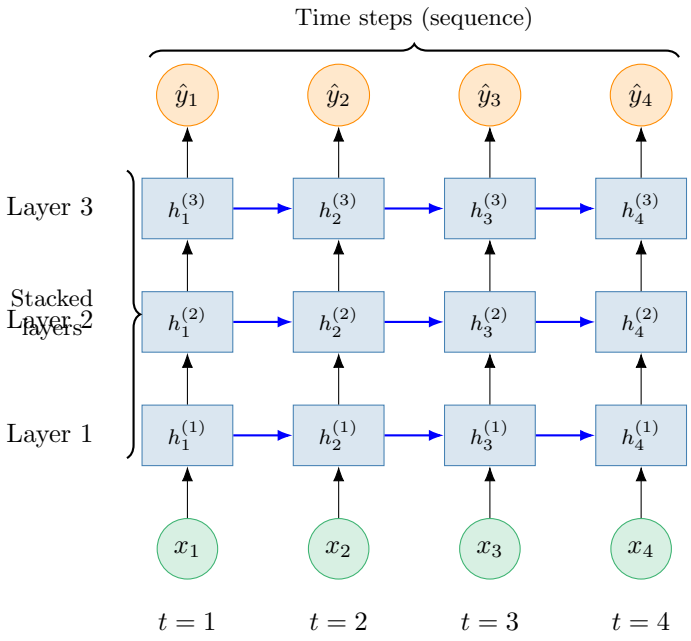


Applications:

- Part-of-speech tagging
- Named entity recognition
- Sentence classification
- Any task where future context is available

Limitation: Cannot be used for generation (no future context available).

5.3 Multi-Layer (Stacked) RNNs



Architecture details:

- **Horizontal arrows (blue):** Recurrent connections within each layer across time
- **Vertical arrows (black):** Connections between layers at each time step
- Each $h_t^{(\ell)}$ receives input from $h_t^{(\ell-1)}$ (layer below) and $h_{t-1}^{(\ell)}$ (previous time)
- Lower layers extract low-level features
- Higher layers capture higher-level abstractions
- Typical depth: 2–4 layers for LSTMs (vs. 50+ for modern transformers)

6 Applications of RNNs

1. **Sequence Tagging**
   - Part-of-speech tagging
   - Named entity recognition
   - Semantic role labeling
2. **Sentence Classification**
   - Sentiment analysis
   - Topic classification
   - Intent detection
3. **Sentence Encoding**
   - Use final hidden state: h_T
   - Or aggregate all states: $\max_t(h_t)$ or $\frac{1}{T}\sum_t h_t$
4. **Conditional Language Models**
   - Speech recognition
   - Machine translation
   - Summarization
   - Dialogue systems

7 Neural Machine Translation

7.1 Machine Translation: Overview

Definition 7.1 (Machine Translation)**. Machine translation (MT)** is the task of translating text from a source language to a target language:

$$\text{Source: } x_1, x_2, \ldots, x_n \quad \longrightarrow \quad \text{Target: } y_1, y_2, \ldots, y_m$$

7.2 Historical Approaches

7.2.1 Rule-Based MT (1950s–1990s)

- Hand-crafted translation rules
- Bilingual dictionaries
- Grammar transformation rules
- **Problems:** Labor-intensive, brittle, low coverage

7.2.2 Statistical MT (1990s–2014)

Based on Bayes' rule decomposition:

$$\hat{y} = \arg\max_y P(y \mid x) = \arg\max_y P(x \mid y) \cdot P(y)$$

- $P(y)$: **Language model** (target language fluency)
- $P(x \mid y)$: **Translation model** (word/phrase correspondences)

Statistical MT Components

1. **Parallel corpus:** Large collection of (x, y) pairs
2. **Alignment model:** Learn word/phrase correspondences
3. **Translation model:** $P(x|y)$ from aligned phrases
4. **Language model:** $P(y)$ for target fluency
5. **Decoder:** Search for best translation

Challenges:

- Complex reordering phenomena
- Multiple separate components (not end-to-end)
- Feature engineering intensive
- Limited context modeling

Example 7.1 (Translation Difficulty)**.** Consider translating Chinese to English:

Chinese: 1519,600,

Correct: In 1519, 600 Spaniards landed in Mexico to conquer the Aztec empire with a population of a few million.

Google Translate (2009): 1519, 600 Spaniards landed in Mexico, millions of people, to conquer the Aztec empire.

The particle indicates modification, but statistical MT failed to capture this structural relationship.

7.3 Neural Machine Translation (NMT)

Definition 7.2 (Neural Machine Translation)**. Neural Machine Translation (NMT)** is a machine translation approach using a single end-to-end neural network trained to maximize:

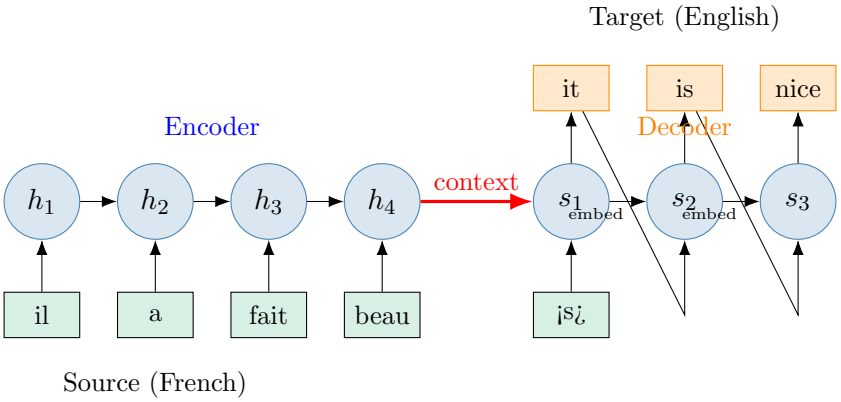
$$P(y \mid x) = \prod_{t=1}^{m} P(y_t \mid y_1, \ldots, y_{t-1}, x_1, \ldots, x_n)$$

Key Innovation: Single neural network replaces entire SMT pipeline.

7.3.1 Sequence-to-Sequence Architecture

The foundational NMT architecture consists of:

1. **Encoder:** RNN that processes source sentence
2. **Decoder:** RNN that generates target sentence



7.3.2 Encoder

The encoder is a (bidirectional) LSTM that processes the source sentence:

$$h_t = \text{LSTM}_{\text{enc}}(x_t, h_{t-1}) \tag{12}$$

$$c = h_n \quad \text{(or a function of all } h_t\text{)} \tag{13}$$

The final hidden state c is the **context vector** or **thought vector** representing the entire source sentence.

7.3.3 Decoder

The decoder is an LSTM language model conditioned on the context:

$$s_0 = c \quad \text{(initialize with context)} \tag{14}$$

$$s_t = \text{LSTM}_{\text{dec}}(y_{t-1}, s_{t-1}) \tag{15}$$

$$P(y_t \mid y_{<t}, x) = \text{softmax}(W_s s_t + b) \tag{16}$$

Training:

- Maximize log-likelihood: $\mathcal{L} = \sum_{t=1}^{m} \log P(y_t^* \mid y_{<t}^*, x)$
- Teacher forcing: Feed gold target words y_{t-1}^* (not predictions)
- End-to-end backpropagation through both encoder and decoder

Inference (Decoding):

- Greedy: $y_t = \arg\max_y P(y \mid y_{<t}, x)$
- Beam search: Keep top-k hypotheses at each step

7.4 Advantages of NMT

1. **End-to-end training:** Single objective, no pipeline errors
2. **Better context modeling:** RNN captures long-range dependencies
3. **Better fluency:** Decoder is a strong language model
4. **Distributed representations:** Continuous representations for words/phrases
5. **Simpler:** Single architecture vs. complex SMT pipeline

7.5 Breakthrough Results

- 2014: Sutskever et al. demonstrate competitive performance
- 2015–2016: NMT surpasses SMT on multiple language pairs
- 2016: Google deploys NMT for Google Translate
- Dramatic improvements in translation quality

7.6 Limitations of Vanilla Seq2Seq

1. **Bottleneck problem:** Entire source compressed into fixed-size vector c
    - Information loss for long sentences
    - Context vector becomes a bottleneck
2. **No explicit alignment:** Model doesn't explicitly learn word correspondences
3. **Inference is slow:** Sequential generation prevents parallelization

Remark 7.1. These limitations motivated the development of **attention mechanisms** (Lecture 7), which revolutionized NMT and became foundational to modern transformers.

8 Summary and Key Takeaways

Key Concepts

1. **Vanishing/Exploding Gradients:**
   - Fundamental problem in training deep and recurrent networks
   - Exploding: solved by gradient clipping
   - Vanishing: requires architectural solutions
2. **LSTM Networks:**
   - Introduce explicit memory (cell state) and gating mechanisms
   - Additive updates prevent vanishing gradients
   - Enable learning of long-range dependencies
   - Three gates: forget, input, output
3. **Advanced RNN Architectures:**
   - Bidirectional: utilize past and future context
   - Multi-layer: hierarchical feature learning
   - Residual connections: improve gradient flow in deep networks
4. **Neural Machine Translation:**
   - End-to-end trainable with encoder-decoder architecture
   - Encoder: compress source into context vector
   - Decoder: conditional language model
   - Breakthrough performance over statistical MT

8.1 Looking Ahead

- **Next lecture:** Attention mechanisms to address seq2seq bottleneck
- **Future topics:** Transformer architecture (attention-only, no RNN)
- **Modern landscape:** Transformers have largely replaced LSTMs

8.2 Historical Perspective

1997	LSTM invented (Hochreiter & Schmidhuber)
2000	Forget gate added (Gers et al.)
2014	Seq2seq for NMT (Sutskever et al.)
2015	Attention mechanism (Bahdanau et al.)
2016	Google deploys NMT
2017	Transformer architecture (Vaswani et al.)
2018+	Transformer dominance (BERT, GPT, etc.)

9 Further Reading

9.1 Foundational Papers

1. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

2. Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10), 2451–2471.

3. Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *NeurIPS*.

4. Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP*.

5. He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. *CVPR*.

9.2 Tutorials and Resources

- Understanding LSTM Networks: `https://colah.github.io/posts/2015-08-Understanding-LSTMs/`
- The Unreasonable Effectiveness of Recurrent Neural Networks: Andrej Karpathy's blog
- Stanford CS224N Course Materials: `https://web.stanford.edu/class/cs224n/`